

Working of the Document Agent

The **Document Agent** is a specialized component of AegisAI responsible for processing, understanding, and retrieving information from business documents. It enables users to upload files such as invoices, bills, receipts, PDFs, images, spreadsheets, and voice recordings, allowing the system to extract valuable business information for analysis and decision-making. The Document Agent is activated whenever a user uploads a document or asks a question related to previously uploaded files.

When a document is received, the agent first identifies its file type and selects the appropriate extraction method. Structured files such as CSV and Excel sheets are read using data processing libraries, while unstructured documents like PDFs and images are processed using Optical Character Recognition (OCR) to extract text. Audio files are converted into text using Speech-to-Text (STT) technology. After extraction, the document content is cleaned, divided into smaller text chunks, and converted into vector embeddings using an embedding model. These embeddings are stored in a local vector database, enabling fast semantic search.

When a user submits a query, the Document Agent converts the query into an embedding and compares it with the stored document vectors using similarity search. The most relevant document sections are retrieved and passed to the Master Coordinator or the appropriate business agent as contextual information. If an LLM is available, the retrieved content is transformed into a clear, natural-language response; otherwise, predefined templates are used. This workflow allows AegisAI to answer document-related queries accurately, summarize uploaded files, and provide context-aware business insights from enterprise documents.

```
import os
import pickle
import numpy as np

class VectorStore:
    def __init__(self, filepath: str = None):
        self.filepath = filepath
        self.documents = [] # List of dicts: {"text": str, "embedding":
list, "metadata": dict}

    def add_document(self, text: str, embedding: list, metadata: dict =
None):
        """Adds a single document chunk and its embedding to the store."""
        self.documents.append({
```

```

        "text": text,
        "embedding": embedding,
        "metadata": metadata or {}
    })

    def similarity_search(self, query_embedding: list, k: int = 3) ->
list:
    """
    Performs a cosine similarity search between the query embedding
and stored documents.
    Returns top k matches: list of (doc_dict, similarity_score)
    """
    if not self.documents:
        return []

    # Convert to numpy arrays
    query_vec = np.array(query_embedding)
    doc_embeddings = np.array([doc["embedding"] for doc in
self.documents])

    # Compute cosine similarity
    # Cos(theta) = A . B / (||A|| ||B||)
    dot_products = np.dot(doc_embeddings, query_vec)
    doc_norms = np.linalg.norm(doc_embeddings, axis=1)
    query_norm = np.linalg.norm(query_vec)

    # Avoid divide by zero
    norms = doc_norms * query_norm
    norms[norms == 0] = 1e-9

    similarities = dot_products / norms

    # Get top k indices sorted in descending order
    top_k_indices = np.argsort(similarities)[::-1][:k]

    results = []
    for idx in top_k_indices:
        results.append({
            "document": self.documents[idx],
            "score": float(similarities[idx])
        })

```

```

        })
    return results

def save(self, filepath: str = None):
    """Saves the index to disk."""
    target_path = filepath or self.filepath
    if not target_path:
        raise ValueError("No filepath specified to save the vector
store.")

    # Ensure directories exist
    os.makedirs(os.path.dirname(os.path.abspath(target_path)),
exist_ok=True)
    with open(target_path, 'wb') as f:
        pickle.dump(self.documents, f)
    print(f"Saved vector store to {target_path} with
{len(self.documents)} items.")

def load(self, filepath: str = None):
    """Loads the index from disk."""
    target_path = filepath or self.filepath
    if not target_path:
        raise ValueError("No filepath specified to load the vector
store.")

    if os.path.exists(target_path):
        with open(target_path, 'rb') as f:
            self.documents = pickle.load(f)
        print(f"Loaded vector store from {target_path} with
{len(self.documents)} items.")
    else:
        self.documents = []
        print(f"No vector store found at {target_path}. Created new
index.")

```